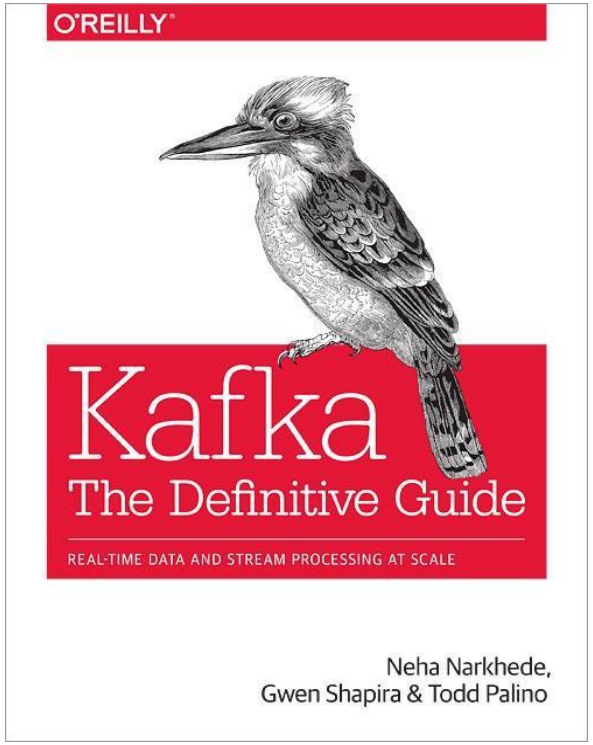
Tema 6. Apache Kafka

PER 14624 Noviembre 2025

Objetivos del tema

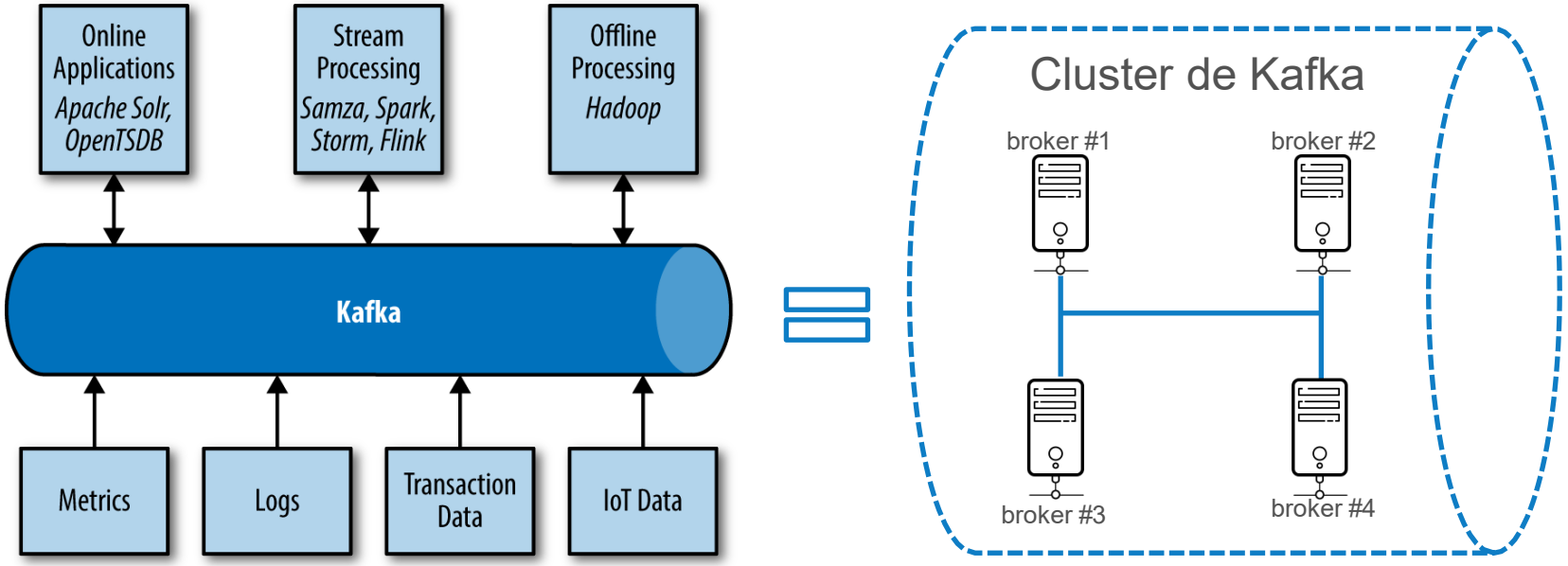
- Entender el concepto de bus de mensajes distribuido y sus casos de uso.
- Conocer los conceptos fundamentales y el funcionamiento interno de Kafka.
- Utilizar Kafka desde Spark, como se verá en la actividad entregable.

kafka


O'REILLY
Kafka
The Definitive Guide
REAL-TIME DATA AND STREAM PROCESSING AT SCALE
Neha Narkhede,
Gwen Shapira & Todd Palino

Apache Kafka

- Surgió como herramienta interna de **LinkedIn** en 2010: mensajería.
- Resuelve la necesidad de tener un *bus de datos común* para poder comunicar aplicaciones que necesitan enviarse datos unas a otras.
- Varias aplicaciones emisoras y receptoras, cada una interesada solo en ciertos tipos de mensajes. Ineficiente conectarlas directamente 2 a 2.
- **Kafka es un bus de datos (cola de mensajes) distribuido y replicado, basado en el paradigma *publicación/suscripción*.**



Apache Kafka

- **Mensaje** = array de bytes sin significado. Equivale a un *registro* en una BD.
- Clave asociada al mensaje: array de bytes. Su hash indica la partición.
- **Topics**: equivale a tabla de una base de datos. Agrupación de mensajes con la misma estructura que pueden interpretarse igual.
  - Al leer o escribir de Kafka, debemos indicar en qué topic.
- Dentro de cada topic se definen **particiones**: distribución física de los datos. Equivalentes a los bloques físicos en los que HDFS divide un archivo.
  - Los mensajes se insertan y se leen por orden de llegada en una misma partición, pero no existe un orden entre particiones distintas de un topic.
- **Distribuido y replicado**: los mensajes se reciben y escriben en el disco local de las máquinas que forman un clúster de Kafka
  - Operan como un único bus de datos.
  - Cada partición es **replicada** automáticamente en varias máquinas: robustez y también un aumento de rendimiento y escalabilidad cuando muchas aplicaciones leen y/o escriben a la vez.
  - Las operaciones de lectura/escritura son directamente **al disco local de cada máquina del cluster**.

Casos de uso de Kafka

- **Tracking de actividad en una web**: propósito original de Kafka en LinkedIn. El frontal web envía mensajes a Kafka (clicks del usuario, modifica su perfil)
  - Consumidos por otros sistemas: monitorización, informes, modelos de ML, o personalizan la página o los resultados en tiempo real
- **Mensajería**: varias aplicaciones envían mensajes que después otra aplicación reúne (consume) y formatea para mandar emails o alertas SMS.
- **Métricas y logs**: múltiples aplicaciones envían logs que luego son consumidos para informes/análisis en modo batch, o alertas en tiempo real
- **Historial de cambios** de BBDD: replicar cambios de una base de datos en otros lugares, o mezclar los cambios desde varias bases de datos distintas.
- **Procesamiento en tiempo real** de flujos de datos: es el caso más típico.
  - Se opera con datos según van llegando para realizar o actualizar agregaciones. Está incluido en los casos anteriores.
  - Frameworks de procesamiento distribuido como Spark, Flink, etc.

Replicación de particiones

- Cada **partición** de un topic está **replicada** en varios brokers: existe un **líder** de la partición, a quien van dirigidas todas las operaciones lectura/escritura
  - El resto de réplicas son followers y se mantienen sincronizadas.

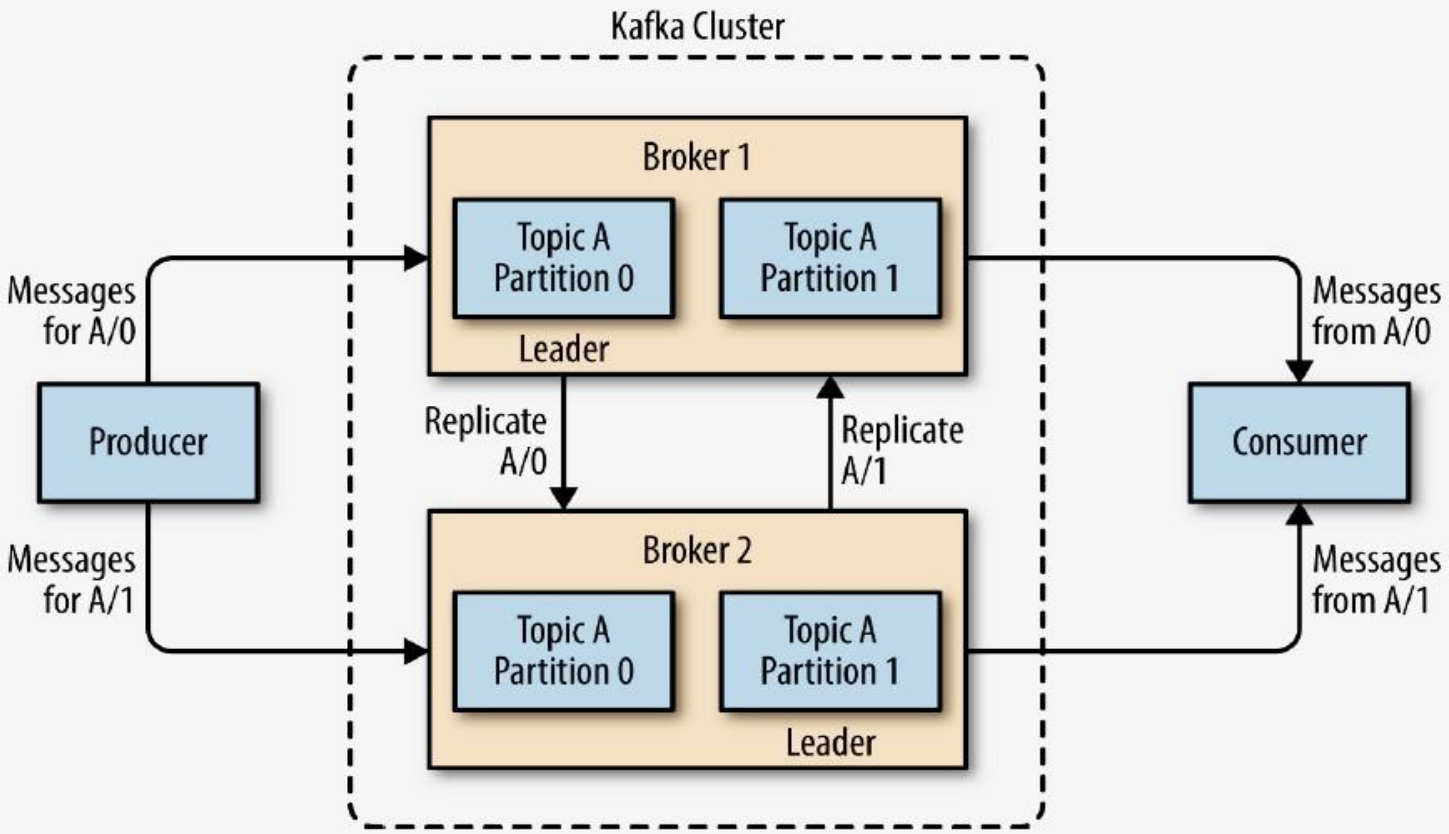


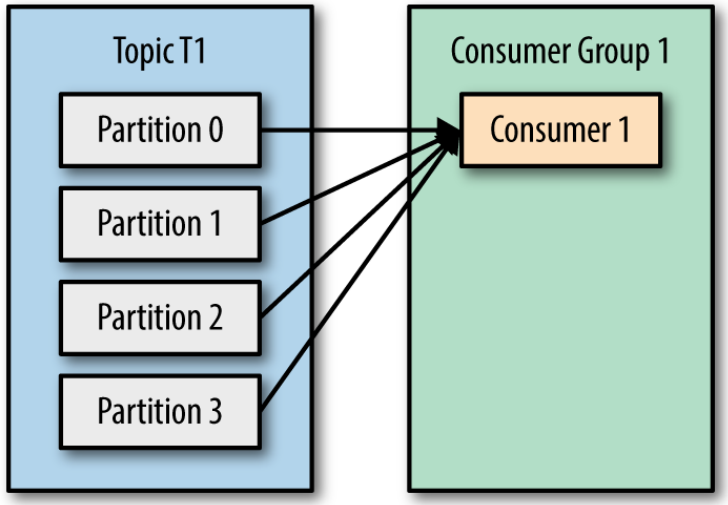
Figura 3. Topic con dos particiones, replicadas. Cada *broker* es líder de una partición.
Fuente: Narkhede, Shapira y Palino, 2017.

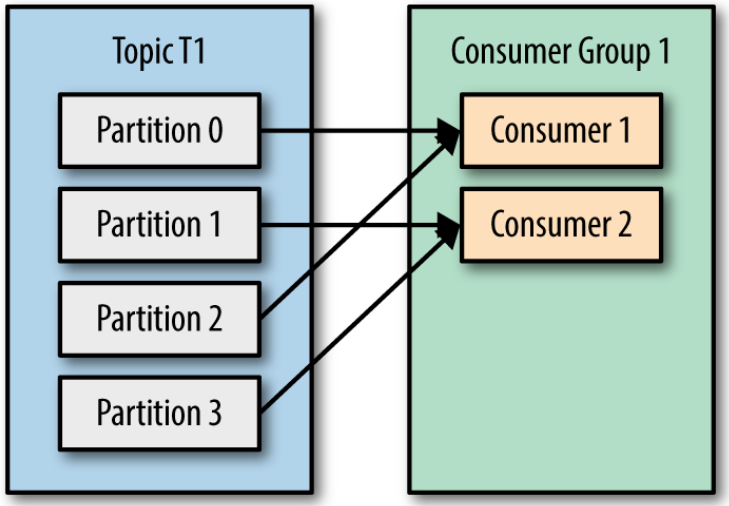
Grupos de consumidores

- Los mensajes están organizados en particiones, físicamente distribuidas entre los brokers.
- Cuando un consumidor lee de un topic, puede leer de cualquier partición.

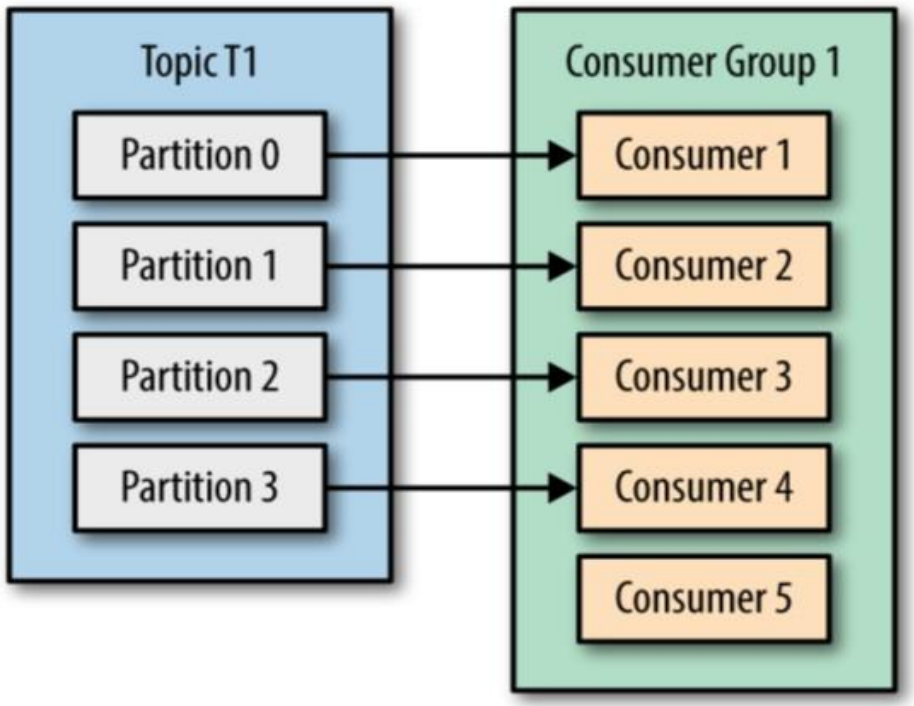
Grupos de consumidores

- Para poder consumir mensajes en paralelo de un mismo topic
- Un proceso consumidor forma parte de un grupo de consumidores concreto. Las particiones existentes en el topic son asignadas por Kafka a distintos consumidores dentro del mismo grupo de consumidores





Grupos de consumidores



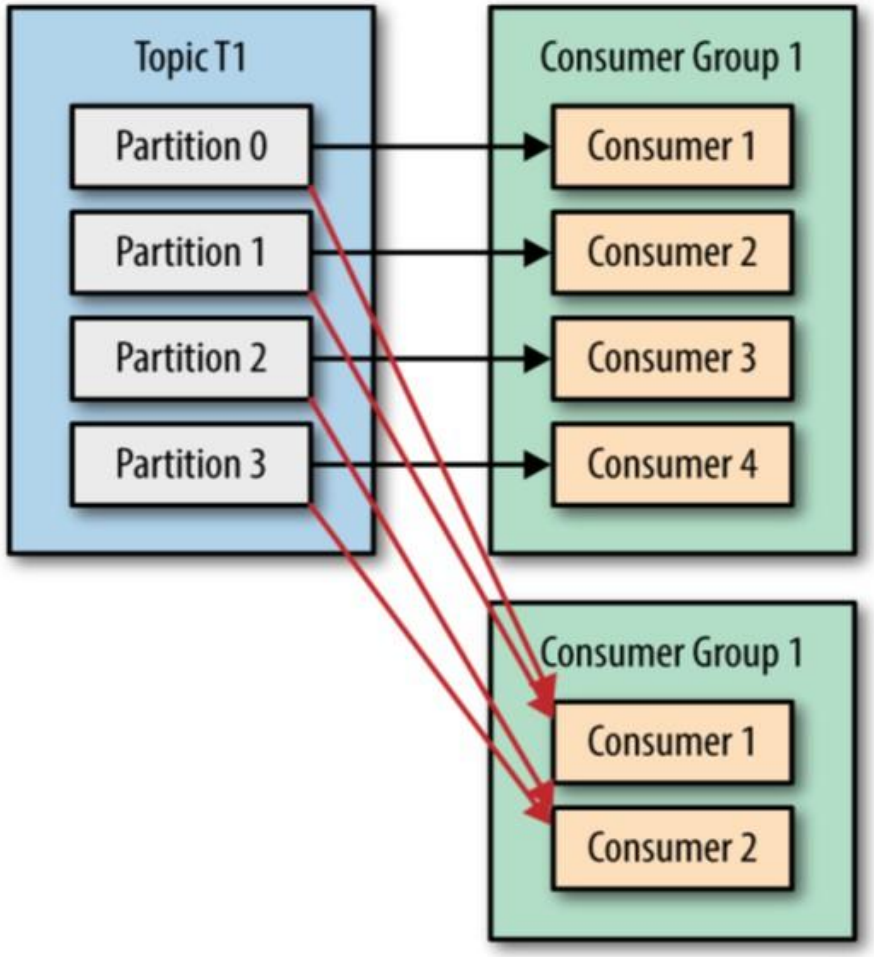


Fig. 6. Izqda: un consumidor ocioso. Dcha: reparto de particiones cuando hay dos grupos de consumidores (p.ej. dos aplicaciones diferentes, independientes).

Proceso de escritura y proceso productor

- El líder de una partición recibe un mensaje para ser escrito en Kafka,
  - ¿El usuario que lo envía tiene permiso para escribirlo en ese topic?
  - ¿Cuántas réplicas de esa partición han de recibir el mensaje para considerarlo como *escrito con éxito*?
    - Parámetro *nacks* de la escritura: 0, 1, all
  - El líder de la partición escribe inmediatamente, y devuelve respuesta si nacks = 0 ó 1. Si no, escribe mensajes en su *purgatorio* hasta que las réplicas le pidan actualizarse y entonces se lo irá enviando a cada una.

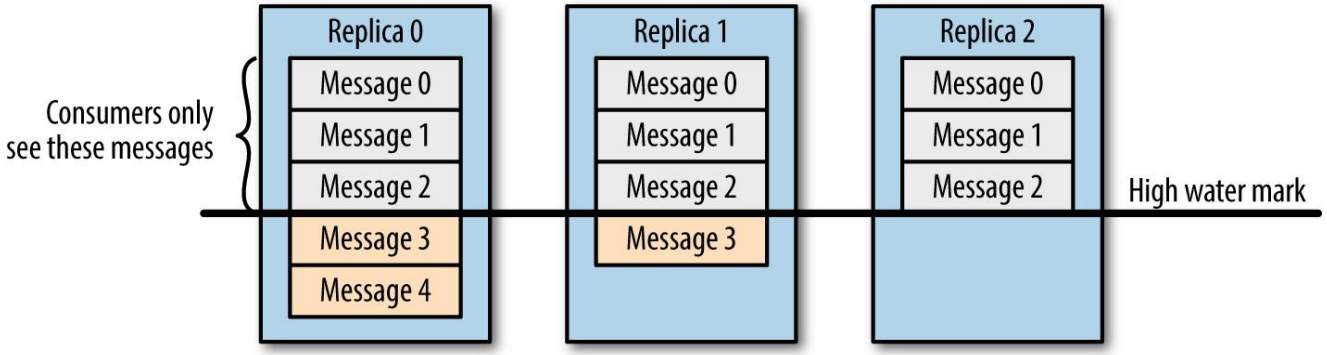
- Para escribir: API de Kafka (en Java) con clase **KafkaProducer** a un topic.
  - Método kafkaProducer.produce() al que se le pasa un objeto mensaje con (clave, valor), que Kafka serializa como array de bytes.
  - **La clave determina la partición** a la que va (mediante código hash)

```
ProducerRecord<String, String> record = new ProducerRecord<>(
            "CustomerCountry", "Biomedical Materials", "USA");

producer.send(record, new DemoProducerCallback()); // enviamos!
```

Proceso de lectura

- **Peticiones de lectura:** envía al líder de la partición una petición indicando el topic y el *offset* del mensaje a partir del cual desea consumir, así como el tamaño máximo del bloque de mensajes que desea que le sean devueltos.
- Si ese offset existe (no se ha borrado aún de disco), el líder de la partición envía directamente desde disco los mensajes solicitados.
  - Acceso rápido gracias a la estructura del fichero y a índices.
  - Kafka utiliza un mecanismo de *zero-copy* para enviar datos directamente dese disco sin pasar por un buffer en memoria principal.
- El cliente indica nº mínimo de mensajes que quiere leer, y t.max de espera
- Sólo se devuelven mensajes que estén replicados en el número de brokers especificados cuando se produjo la petición de escritura del mensaje



Dudas y preguntas

¡MUCHAS GRACIAS¡

unir
LA UNIVERSIDAD
EN INTERNET